

# Distributed programming in Spark

## II.3502 - Distributed architectures and programming: Lab 5

isep

# Table of Contents

1 Spark Deployment mode

2 Installing Spark

3 Spark applications

4 Task and Deliverables

# Table of Contents

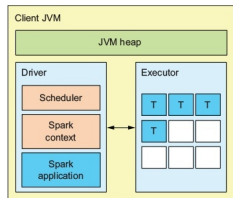
1 Spark Deployment mode

2 Installing Spark

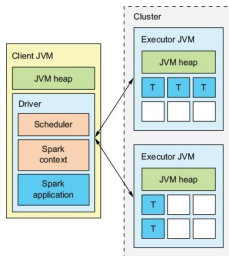
3 Spark applications

4 Task and Deliverables

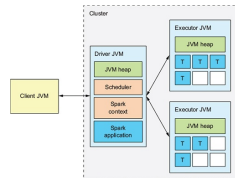
# Spark Execution modes



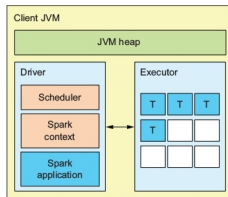
Local mode



Cluster mode



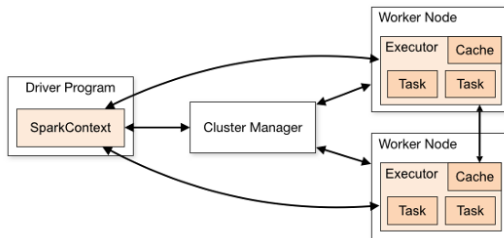
# Spark Execution - Local



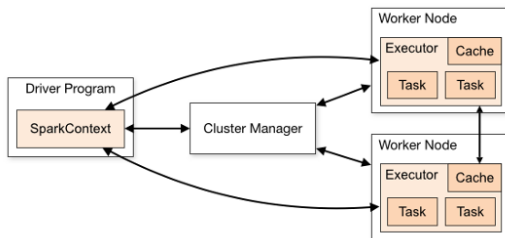
- The entire processing is done on a single (non-distributed) JVM.
- Spark spawns all the execution components - driver, executor, LocalSchedulerBackend, and master - in the same single JVM.
- Parallelisation is ensures across all the cores in your server.
- The default parallelism is the number of threads as specified in the master URL.

# Spark Execution - Cluster

- Standalone Deploy Mode
- Apache Mesos
- Hadoop YARN
- Kubernetes



# Spark Execution - Cluster

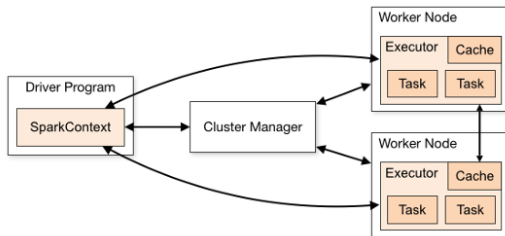


- Spark applications are run as independent sets of processes, coordinated by a *SparkContext* in a **Driver**<sup>1</sup> program.
- The *context* connects to the **Cluster Manager** which allocates resources.
- Each **Worker** in the cluster is managed by an executor.
- The **Executor** manages *Task* computation as well as *storage* and caching on each Worker node.

---

<sup>1</sup>Process running the `main()` function of the application and creating the *SparkContext*

# Spark Execution - Cluster

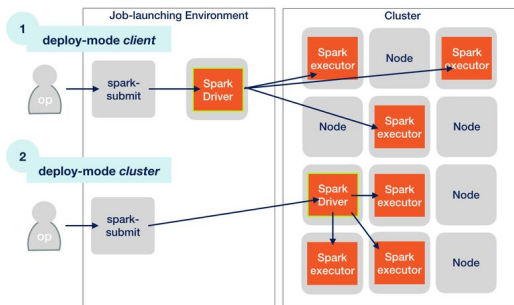


- **Spark App:** Each SparkContext creates a Spark application, which includes scheduling components. Upon **Action** the Driver program submits the job to the Cluster Manager.
- **Cluster Manager:** Start Executors on Worker Nodes.
- **Worker:** Launch Spark Executor in a process. Tasks are launched in separate threads, one per each core on the Worker Node (configurable).

# Spark - Standalone cluster - Deploy mode

Spark deployment mode (`--deploy-mode`) specifies where to run the driver program of your Spark application/job.

Spark provides two deployment modes, `client` and `cluster`.



# Spark - Standalone cluster - Deploy mode

## Client deploy mode (default)

In client mode, the **Spark driver** component of the spark application will run on the machine from where the job submitted.

```
# client deployment mode usage
spark-submit --deploy-mode client --driver-memory xxxx .....
```

- The client machine is responsible for managing the driver program, including task scheduling and execution.
- Provides direct access to the driver program's logs and output.
- Client mode is used for debugging/testing purposes (not for production).
- If a machine or a user session running spark-submit terminates, application also terminates with status fail.

# Spark - Standalone cluster - Deploy mode

## Cluster deploy mode <sup>1</sup>

In cluster mode, the **driver** runs on one of the nodes within the Spark cluster.

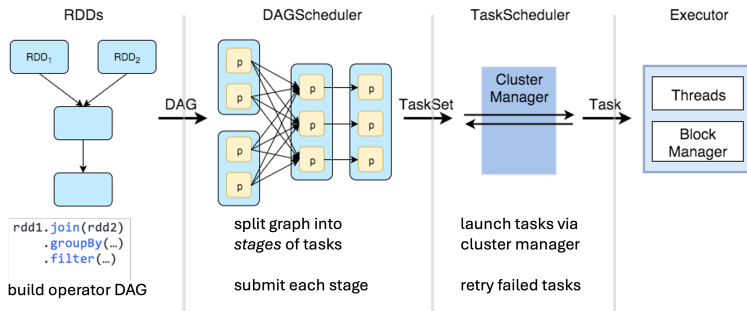
```
# cluster deployment mode usage
spark-submit --deploy-mode cluster --driver-memory xxxx .....
```

- The Spark cluster manager (like YARN or Mesos) allocates resources and manages the driver and executor processes.
- Suitable for production deployments, where the Spark application needs to utilize the full resources of the cluster efficiently (for large dataset processing).

---

<sup>1</sup>Cluster deploy mode is not applicable to Spark shells. `pyspark` or `spark-shell` (CLI or via notebooks) does not support the `cluster` mode. If you are running an interactive shell, by default you are running in `client` mode.

# Spark Job scheduling



# Table of Contents

- 1 Spark Deployment mode
- 2 Installing Spark**
- 3 Spark applications
- 4 Task and Deliverables

# Installing Spark

## 1. Prerequisites:

- **Java** (better JDK 8)

```
java -version
```

- **Python** (for PySpark)

```
python --version
```

- **Scala** (optional for Spark Shell)

```
scala -version
```

- Hadoop (optional) for running on top of HDFS

# Installing Spark

## 2. Install Apache Spark

- Manually:

Download Spark: <https://spark.apache.org/downloads.html>

- ▶ Linux/MacOS:

- ```
tar -xzf spark-<version>-bin-hadoop<version>.tgz
```

- move to a preferred directory

- ▶ Windows:

- extract `.tgz` and move to a directory (e.g., `C:\spark`)

- Use package managers to install Spark: Homebrew or APT (Debian/Ubuntu)
- install PySpark with `conda` or `pip`

# Installing Spark

## 3. Set Environment Variables:

- Linux/MacOS

- add `SPARK_HOME` and `JAVA_HOME` to your `~/.bashrc` or `~/.zshrc` file
- reload the shell: `source ~/.bashrc`
- update `PATH` variable

- Windows

- open `System Properties` → `Environment Variables`
- add new variables: `SPARK_HOME` and `JAVA_HOME`
- update `PATH` variable

# Installing Spark

4. Verify the installation by running:

- `pyspark`
- `spark-shell`

# Table of Contents

- 1 Spark Deployment mode
- 2 Installing Spark
- 3 Spark applications**
- 4 Task and Deliverables

# Word count application

## PySpark

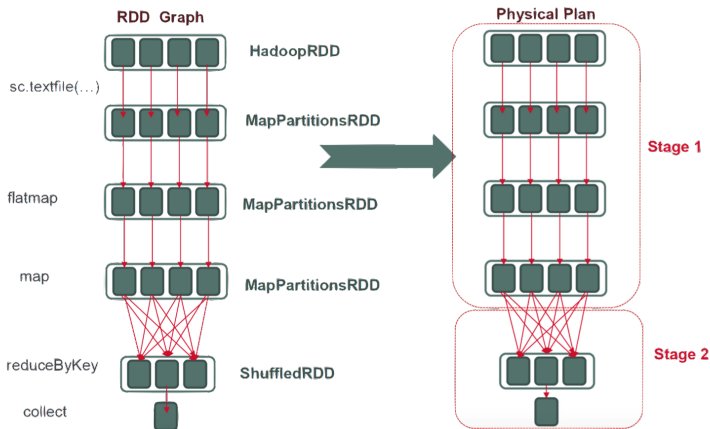
### wordcount.py

- Performs a `textFile` operation to read an input file (local or HDFS)
- `flatMap` operation to split each line into words
- `map` operation to emit `(word, 1)` pairs
- `reduceByKey` operation to sum the counts (all the 1's) for each word

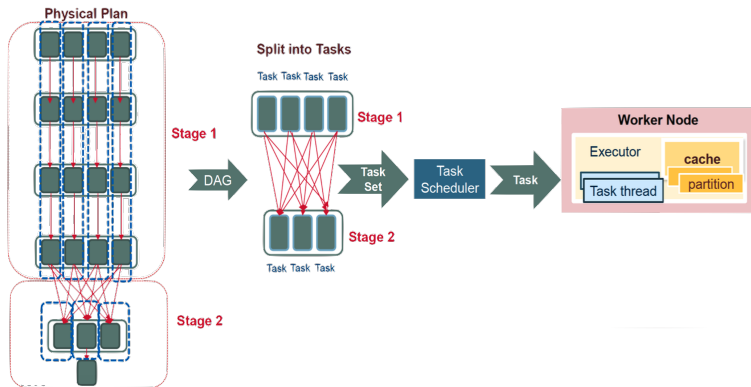
```
import sys
from pyspark import SparkContext, SparkConf

if __name__ == "__main__":
    conf = SparkConf().setAppName("Spark-Count")
    sc = SparkContext(conf=conf)
    inputFile = sys.argv[1]
    textFile = sc.textFile(inputFile)
    wordCounts = textFile.flatMap(lambda line: line.split()).\
        map(lambda word: (word, 1)).reduceByKey(lambda a, b: a+b)
    output=wordCounts.collect()
    for (word, count) in output:
        print("%s: %i" % (word, count))
```

# Execution Plan



# Execution Plan



Operations that can run on the same partition are executed in stages

# Running Spark Applications

**Spark shells** are great for developing and testing:

- quickly introduction to using Spark
- simple way to learn API
- execute and test Spark commands
- minimal setup, preloaded SparkContext/SparkSession, and support for multiple languages (Scala, Python, R)

**Spark-submit** jobs are more likely to be used in production workflows.

# Running Spark - Interactive shells

`pyshell`

**PySpark** shell is an interactive interface for using Apache Spark with Python

- Install Spark and Python on your machine.
- set `SPARK_HOME` and add its `bin` directory to your system's `PATH`
- type `pyspark` in the terminal
- to customize resources (memory, cores, etc.), provide options:  
`pyspark --master local[2] --driver-memory 2g`
- exiting pyspark: `exit()`

# Running Spark - Interactive shells

`spark-shell`

**Spark Shell** is an interactive environment for running Apache Spark commands in Scala.

- Install Spark on your machine.
- set `SPARK_HOME` and add its `bin` directory to your system's `PATH`
- type `spark-shell` in the terminal
- to customize resources (memory, cores, etc.), provide options:  
`spark-shell --master local[2] --driver-memory 2g`
- exiting spark-shell: `:exit`

# Running Spark - Interactive shells

Key components in Spark Shells:

- **SparkContext** `sc`: Entry point to interact with Spark functionality
- **SparkSession** `spark`: Unified entry point for DataFrame and Dataset APIs
- to check deploy mode:
  - ▶ Python: `sc.getConf().get("spark.submit.deployMode")`
  - ▶ Scala: `sc.deployMode`

## --deploy-mode

If you are running an interactive shell, e.g. `pyspark` (CLI or notebook), by default you are running in `client` mode.

Error: Cluster deploy mode is not applicable to Spark shells.

# Running Spark with Jupyter Notebook

- Install Spark and Jupyter
- set `SPARK_HOME` and add its bin directory to your system's `PATH`
- starting a Jupyter session in the same terminal window

Or you can configure PySpark to work with Jupyter by:

- setting env vars in `~/.bashrc` or `~/.zshrc` file:

```
export PYSPARK_DRIVER_PYTHON=jupyter
export PYSPARK_DRIVER_PYTHON_OPTS='notebook'
```

- launch PySpark in Jupyter: `pyspark`

Check and run '*Hands-On PySpark.ipynb*' file, available on Moodle.



# Spark API's

## wordcount with Scala

```
import org.apache.spark.{SparkConf, SparkContext}

object WordCount {
  def main(args: Array[String]): Unit = {
    val conf = new SparkConf().setAppName("Spark-Count")
    val sc = new SparkContext(conf)

    val inputFile = args(0)
    val textFile = sc.textFile(inputFile)

    val wordCounts = textFile
      .flatMap(line => line.split("\\s+"))
      .map(word => (word, 1))
      .reduceByKey(_ + _)

    val output = wordCounts.collect()
    output.foreach { case (word, count) =>
      println(s"$word: - $count")
    }

    sc.stop()
  }
}
```

# Spark API's

## wordcount with Java and Maven

- create Maven project with the WordCount.java class (attached to the lab<sup>2</sup>)
- add dependencies in pom.xml (attached)
- build the project with `mvn clean package`
- locate the `.jar` file with dependencies in `target` folder
- Run application:

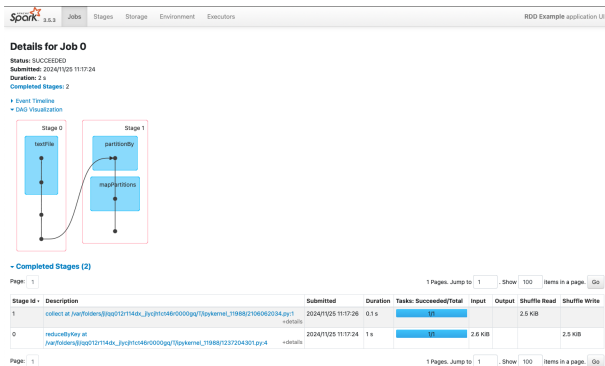
```
spark-submit --class org.example.WordCount \  
wordcount-spark/target/wordcount-spark-1.0-SNAPSHOT-jar-with-dependencies.jar \  
some_text.txt res_java
```

---

<sup>2</sup>WordCount class and Maven dependencies are available on Moodle

# Monitoring Spark and applications

## Driver Spark UI



Every SparkContext launches a web UI (**Spark driver's web UI**), by default on port **4040**<sup>3</sup>

<sup>3</sup>check spark logs for host and port. Spark context Web UI available at [http://\[host\]:4040](http://[host]:4040) (i.e for local deployment access url is <http://localhost:4040>)

# Table of Contents

- 1 Spark Deployment mode
- 2 Installing Spark
- 3 Spark applications
- 4 Task and Deliverables**

# Data Processing application

Implement a Spark application using RDD-based transformations and aggregations to analyze COVID-19 data.

- Load data: Use the COVID-19 dataset from Our World in Data: `owid-covid-data.csv`
- Clean data:
  - ▶ Parse each line to extract relevant fields (e.g., `location`, `date`, `total_cases`, `total_deaths`, `people_fully_vaccinated`, `population`).
  - ▶ Filter out rows with missing or invalid values.
- Data transformation: calculate vaccination rates for each country; Map data to key-value pairs for easy aggregation
- Aggregations:
  - ▶ Compute continent-level daily averages for new cases and deaths
  - ▶ Calculate total cases, deaths, and vaccination rates for each country
  - ▶ Identify the top 10 countries with the highest vaccination rates
- Store aggregated results to the file system (e.g. `csv` or `txt` files)

# Deliverables

- Running hands-on examples with API chosen
- Data processing application using Apache Spark
  - ▶ Aggregated results saved to files
  - ▶ Documented code explaining each step
- Execution details: running the application locally or on a cluster (e.g., Hadoop, Spark standalone), deployment mode, Drivers web UI screenshots while running app (DAG, Jobs).